

Software Testing and Quality Assurance

Mod 4: Test Automation and Testing for Specialized Environment

Software quality

Software quality in software testing and quality assurance refers to the degree to which a software product meets specified requirements, satisfies user needs, and is free of defects. It embodies multiple attributes such as reliability, usability, performance, security, maintainability, and portability. Software quality is assessed through both the product itself and the processes used to develop it, ensuring that the software is robust, reliable, and meets business objectives. Key characteristics of software quality include:

- **Functional suitability:** Software meets the functional requirements under specific conditions.
- **Reliability:** Software operates without failure.
- **Performance efficiency:** Software performs well in terms of speed, responsiveness, and scalability.
- **Security:** Software is protected against threats.
- **Maintainability:** Software can be modified or extended efficiently.
- **Portability:** Software can operate across different environments or platforms.

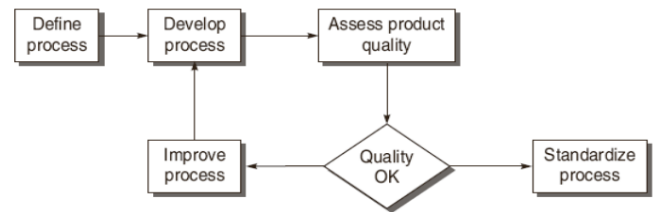
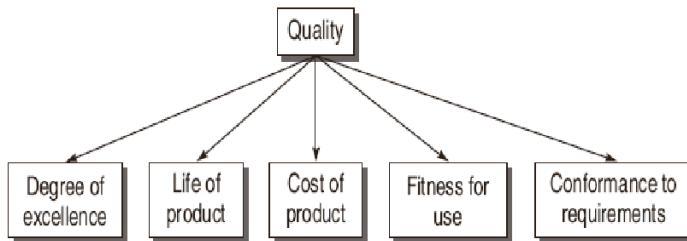


Figure 13.2 Process affects quality

Five views of software quality

1. Transcendental view: Quality is seen as an ideal or absolute concept that is universally recognizable but difficult to define precisely. In this view, quality is an inherent excellence, something that can be recognized intuitively and is often associated with high standards or superior achievement. It is more of a philosophical or subjective concept that cannot be easily measured but is understood through experience.

2. User view: Quality is defined in terms of fitness for purpose or usefulness to the user. It emphasizes whether the software satisfies the needs, expectations, and preferences of its users. Here, quality is judged by the user's experience and satisfaction, focusing on usability, effectiveness, and how well the software meets the intended use.

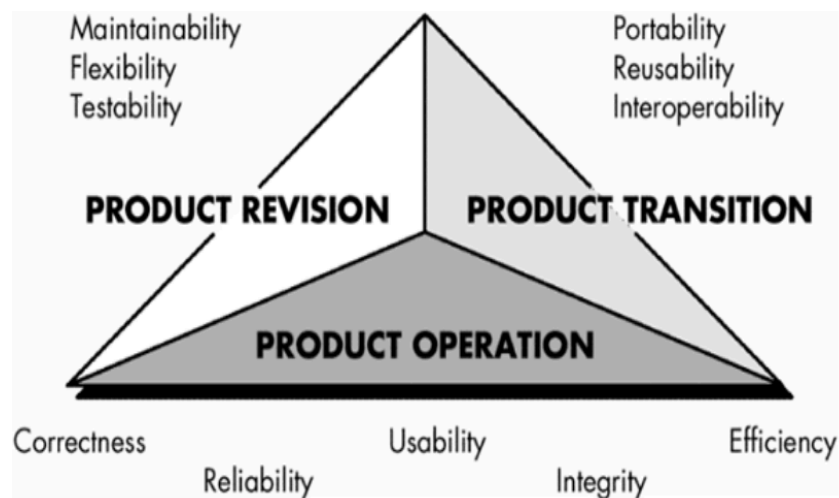
3. Manufacturing view: Quality is defined as conformance to specifications or requirements. From this perspective, quality means that the software product complies exactly with the design specifications without deviations. This view is focused on the production process and defect prevention, ensuring the product is built right from the beginning.

4. Product view: Quality is associated with the inherent characteristics and measurable attributes of the software product itself, such as reliability, maintainability, security, and performance. This view emphasizes the internal properties of the software that determine its external quality. The product's structural and functional characteristics are the basis for assessing quality objectively.

5. Value-based view: Quality is defined by the value or worth of the software product from the customer's perspective, often related to the price they are willing to pay and the benefits they receive. This perspective considers the balance between the cost of the product and the advantages it offers, reflecting economic considerations.

McCall's quality factors and criteria

Quality Factors	Definitions
Correctness	The extent to which a program satisfies its specifications and fulfills the user's mission objectives.
Reliability	The extent to which a program can be expected to perform its intended function with required precision.
Efficiency	The amount of computing resources and code required by a program to perform a function.
Integrity	The extent to which access to software or data by unauthorized persons can be controlled.
Usability	The effort required to learn, operate, prepare input, and interpret output of a program.
Maintainability	The effort required to locate and fix a defect in an operational program.
Testability	The effort required to test a program to ensure that it performs its intended functions.
Flexibility	The effort required to modify an operational program.
Portability	The effort required to transfer a program from one hardware and/ or software environment to another.
Reusability	The extent to which parts of a software system can be reused in other applications.
Interoperability	The effort required to couple one system with another.



Quality Categories	Quality Factors	Broad Objectives
Product Operation	Correctness Reliability Efficiency Integrity Usability	Does it do what the customer wants? Does it do it accurately all of the time? Does it quickly solve the intended problem? Is it secure? Can I run it?
Product Revision	Maintainability Testability Flexibility	Can it be fixed? Can it be tested? Can it be changed?
Product Transition	Portability Reusability Interoperability	Can it be used on another machine? Can parts of it be reused? Can it interface with another system?

Quality criteria

A quality criterion is an attribute of a quality factor that is related to software development. Example: Modularity is an attribute of the architecture of a software system. A highly modular software allows designers to put cohesive components in one module, thereby increasing the maintainability of the system.

Quality Criteria	Definitions of Quality Criteria
Access audit	The ease with which software and data can be checked for compliance with standards or other requirements.
Access control	The provisions for control and protection of the software and data.
Accuracy	The precisions of computations and output.
Communication commonality	The degree to which standard protocols and interfaces are used.
Completeness	The degree to which a full implementation of the required functionalities has been achieved.
Communicativeness	The ease with which inputs and outputs can be assimilated.
Conciseness	The compactness of the source code, in terms of lines of code.
Consistency	The use of uniform design and implementation techniques and notations throughout a project.
Data commonality	The use of standard data representations.
Error tolerance	The degree to which continuity of operation is ensured under adverse conditions.
Execution efficiency	The run-time efficiency of the software.
Expandability	The degree to which storage requirements or software functions can be expanded.
Generality	The breadth of the potential application of software components.
Hardware independence	The degree to which the software is dependent on the underlying hardware.
Instrumentation	The degree to which the software provides for measurements of its use or identification of errors.
Modularity	The provision of highly independent modules.
Operability	The ease of operation of the software.
Self-documentation	The provision of in-line documentation that explains the implementation of components.
Simplicity	The ease with which the software can be understood.
Software system independence	The degree to which the software is independent of its software environment – non-standard language constructs, operating system, libraries, database management system, etc.
Software efficiency	The run-time storage requirements of the software.
Traceability	The ability to link software components to requirements.
Training	The ease with which new users can use the system.

Relationship between quality factors and quality

Quality factors are attributes or characteristics that influence the overall quality of a software product. They represent specific areas or dimensions in which the software can be evaluated for quality. Quality itself is a broader concept, reflecting the degree to which the software meets requirements, satisfies user needs, and performs reliably and efficiently in its operational environment.

Quality factors break down the abstract notion of quality into measurable or assessable components. By evaluating these individual factors, stakeholders can get a clearer, structured understanding of the software's quality. For example, factors like correctness, reliability, efficiency, usability, and maintainability each contribute to defining the software's overall quality.

In essence, quality factors serve as criteria or indicators that guide the assessment, improvement, and management of software quality. They provide a practical means to measure quality and help shape development and testing efforts to ensure the software meets desired quality standards.

Quality management: Quality management is to:

- Plan suitable quality control and quality assurance activities,
- Define procedures and standards which should be used during software development and verifying that these are being followed by everyone
- Properly execute and control activities.

Software quality metrics

Software quality metrics are a subset of software metrics that focus on the quality aspects of the product, process, and project. These are more closely associated with process and product metrics than with project metrics. Software quality metrics can be further divided into three categories –

- Product quality metrics
- In-process quality metrics
- Maintenance quality metrics

Product quality metrics

Product quality metrics are quantitative measures used to assess the quality attributes of a software product. The following are key product quality metrics:

- **Mean time to failure (MTTF):** Measures the average time the software operates without failure. It indicates the reliability of the software by showing how long it runs before experiencing a breakdown or fault.
- **Defect density:** It is defined as the number of defects found in the software relative to the size of the software (often measured in LOCs or FPs). It helps evaluate the quality by indicating how many defects exist per unit of software, reflecting the error-proneness and overall code quality.
- **Customer problems:** Tracks the number and severity of issues reported by customers while using the software. It provides insight into the real-world performance and usability of the software from the end-user perspective.
- **Customer satisfaction:** It is based on user feedback and surveys to measure how satisfied customers are with the software product. High customer satisfaction indicates that the software meets or exceeds user expectations for functionality, usability, and reliability.

In-process quality metrics

In-process quality metrics focus on monitoring and evaluating software defects and their management during the formal machine testing phase. These metrics provide insight into the efficiency and effectiveness of the testing and development process. Key in-process quality metrics include:

- **Defect density during machine testing:** Measures the number of defects found per unit size of the software (e.g., per 1000 LOC) during formal testing. A higher defect density signals more errors in the code, indicating the quality of the software under test and the development process.
- **Defect arrival pattern during machine testing:** Tracks how defects are reported over time during testing. This pattern helps identify periods of high defect discovery and correlates with testing effectiveness and software quality stability.
- **Phase-based defect removal pattern:** Shows the effectiveness of removing defects in different phases of development (e.g., design, coding, testing). This pattern helps evaluate at what stage defects are being caught and removed, guiding process improvements.
- **Defect removal effectiveness:** Measures the ratio of defects found and fixed during testing compared to defects found after release. High effectiveness indicates good testing and quality control processes, minimizing defects reaching customers.

Maintenance quality metrics

Maintenance quality metrics help monitor and improve the quality of software during the maintenance phase by tracking how defects are managed and fixed. Important metrics include:

- **Fix backlog and backlog management index:** Tracks the number of outstanding defect fixes or change requests that have not yet been resolved. The backlog management index evaluates how well the backlog is being controlled and prioritized, ensuring no critical fixes are delayed excessively.
- **Fix response time and fix responsiveness:** Measures the time taken from when a defect is reported to when the fix process starts. Reflects how quickly the maintenance team responds to reported issues, which impacts customer satisfaction and software reliability.

- **Percent delinquent fixes:** The percentage of fixes that have exceeded the expected or agreed response time criteria. Indicates delays in defect resolution and can highlight bottlenecks or team overload in maintenance.
- **Fix quality:** Assesses the effectiveness of a fix, ensuring that the defect is properly resolved without introducing new defects. Poor fix quality (defective fixes) can cause recurring problems, reducing overall software quality and user trust.

PDCA Cycle



The PDCA cycle is viewed as one of the key concepts in the field of quality management and its consistent development. It is an organized and cyclic method adopted in the improvement of processes as well as goods and services being offered. The cycle consists of four key stages: PDCA (Plan, Do, Check, Act). Through the following stages, it is possible for organizations and people to adapt to minor changes and solve issues in order to achieve better outcomes.

PDCA is an iterative technique that acts as a management method for various business purposes like improvement, control, and continuation of processes and products. In simpler words, it is a cycle for solving problems and improving processes.

The model is a fundamental principle including the scientific method and allows both control and analysis – once the hypotheses are confirmed, executing the cycle on the repeated process extends the knowledge further and brings the goals closer with better operation and outputs.

PDCA consists four steps:

1. **Plan:** A well-defined project plan provides a framework for operations. Importantly, it should reflect the organization's mission and values. It should also map the project's goals and clearly indicate the best way to attain them.
2. **Do:** This is the step where the plan is set in motion. The plan was made for a reason, so it is important for players to execute it as outlined. This stage can be broken down into three sub-segments, including training of all personnel involved in the project, the actual process of doing the work, and recording insights, or data, for future evaluation.
3. **Check:** Typically, there should be two checks throughout the project. First, checks alongside implementation ensure the project's objectives are being met. Second, a more comprehensive review of the project carried out upon completion addresses the successes and failures so that future adjustments can be made.
4. **Act:** The final step is to take corrective action once past mistakes have been identified and resolved. The PDCA Cycle is repeated and can be redefined perhaps to better results under new guidelines.

Quality plan

A quality plan is a structured document that outlines how an organization will ensure software quality throughout the project lifecycle. Its key points are:

- **Defines quality objectives:** Specifies clear quality goals, standards, and expectations for the software product based on stakeholder requirements and business needs.
- **Identifies quality standards:** Lists applicable standards (industry, regulatory, organizational) that the project must adhere to, ensuring consistency and compliance.
- **Clarifies roles and responsibilities:** Assigns specific quality-related tasks and responsibilities to team members, such as developers, testers, managers, and QA leads.
- **Details quality assurance activities:** Describes planned activities like reviews, inspections, audits, and testing for defect prevention and quality validation.
- **Specifies metrics and criteria:** Defines how software quality will be measured (e.g., defect density, customer satisfaction), including target thresholds and acceptable limits.

- **Establishes risk management:** Includes strategies to identify, monitor, and mitigate risks that could affect the quality of the software.
- **Focuses on continuous improvement:** Incorporates mechanisms for feedback, evaluation, and adaptation to improve quality processes throughout the project.

Quality assurance

Quality assurance is a systematic set of activities designed to ensure that processes, procedures, and standards are followed correctly to produce high-quality software products.

- **Focus on process:** QA emphasizes improving and standardizing the software development process rather than just detecting defects in the final product.
- **Prevention-oriented:** It aims at preventing defects by establishing quality policies, setting standards, conducting reviews, and planning effective testing.
- **Key activities:** Includes process definition and implementation, audits, reviews, testing oversight, defect tracking, risk management, and staff training.
- **Standards compliance:** Ensures adherence to organizational and international standards like ISO and IEEE to maintain consistency and quality.
- **Continuous improvement:** QA involves ongoing evaluation and enhancement of software processes to achieve better quality outcomes over time.
- **Roles and responsibilities:** Involves all stakeholders including project managers, developers, testers, and dedicated QA teams to enforce quality practices throughout the software development lifecycle.
- **Tools and techniques:** Uses a combination of manual and automated testing, code inspections, walkthroughs, and metrics analysis to monitor and improve quality.
- **Integration in SDLC:** QA is integrated into every phase of the Software Development Life Cycle (SDLC), ensuring that progress moves to the next phase only when quality criteria are met.

Quality control and methods

Quality Control (QC) in software is a reactive process focused on identifying defects in a developed software product to ensure it meets the required standards and specifications. It involves systematic testing, inspections, and reviews that validate the software's correctness, functionality, and efficiency before release. QC verifies that the product is defect-free, reliable, and safe to use, building user trust and reducing risks of failures in production. Key activities in QC include defect detection, tracking, prioritization, fixing verification, and compliance checks. Unlike Quality Assurance, which focuses on improving processes to prevent defects, Quality Control concentrates on validating the final product's quality by uncovering and addressing issues through validation and testing techniques.

Procedural approach to quality management

1. **Define quality:** Defining what quality means to the organization and anticipating the quality standards and requirements.
2. **Establish processes:** Ideate and implement the processes which will lead to meeting the requirements and quality standards along with responsibilities that should be taken for each step, and how each step will be monitored
3. **Train personnel:** Train personnel to understand the quality goals, standards, and procedures so that they can consistently deliver high-quality products or services.
4. **Monitor and measure:** Monitoring consistently increases the change to ensure that it is operating as intended and desired quality level of the product is being achieved.
5. **Consistent improvement:** Continuously analyze the process data and make improvements as needed to maintain or improve the quality level.

Quantitative approach to quality management

1. **Collect data:** Collect data on the process, product, or service to be analyzed. The data that is related to the process, product, or service is collected for the analysis process.

2. **Analyze data:** To analyze data, Statistical methods are useful to identify the trends, and patterns for the future growth of the organization.
3. **Identify cause-and-effect relationships:** The analyzed data in the data analysis phase is used to identify the cause-and-effect relationships with the issues that affect the quality of the product and identify the root cause of the problem to reduce the repetition of the issue
4. **Implement improvements:** Based on the results from data analysis and Identifying the cause-and-effect relationship, new changes or features are added to increase the quality level by implementing the improvements.
5. **Monitor and measure:** To ensure that the improvements lead to providing the desired quality of the product and satisfying the requirements, regular monitoring along with certain measures is needed.



Quality cost and benefits

Quality cost refers to the total expenses incurred to ensure that a software product meets quality standards and the costs resulting from failing to meet these standards. It is typically divided into four categories:

- **Prevention costs** (investments in training, process improvements to prevent defects)
- **Appraisal costs** (inspection, testing activities to detect defects)
- **Internal failure costs** (costs to fix defects found before release)
- **External failure costs** (costs related to defects found after delivery such as customer support, patches, and reputation damage)

Investing in quality upfront reduces the costly impact of failures downstream.

Benefits of investment on quality:

- Reduces overall defect rates and minimizes costly rework and failures.
- Improves customer satisfaction and trust by delivering reliable software.
- Enhances operational efficiency and compliance with standards.
- Decreases risk of product recalls and legal issues.
- Leads to better market reputation and competitive advantage.
- Optimizes resource allocation and lowers long-term costs through early defect prevention and quality control.

Defect prevention

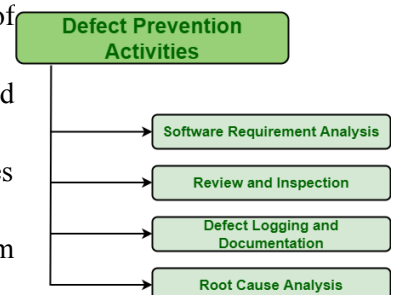
Defect prevention is the proactive process of identifying and eliminating the root causes of defects before they occur in software.

It aims to reduce the number and impact of defects, thereby improving software quality and development efficiency.

Early defect detection and resolution during requirements analysis and design phases significantly lower defect introduction.

Reviews and inspections (self-review, peer review) help find defects early, preventing them from progressing further.

Defect logging and documentation provide a basis for analyzing and understanding defects to avoid recurrence.



Root cause analysis techniques (such as Pareto analysis and Ishikawa diagrams) are used to identify fundamental causes of defects.

Static code analysis and automated testing tools help detect code-level issues early without execution.

Practices like pair programming, test-driven development (TDD), and continuous training improve code quality and prevent defects.

The use of checklists ensures adherence to standards and avoids omissions during development activities.

Root cause analysis: Root Cause Analysis is a systematic problem-solving process used to identify the fundamental cause(s) of a problem or defect. The goal is to find the underlying reasons, not just the symptoms, to implement effective corrective actions. RCA involves collecting data, mapping out timelines or causal graphs, and investigating factors leading to an issue.

Common tools and techniques include the 5 Whys method, Ishikawa (fishbone) diagrams, failure mode and effects analysis (FMEA), and Pareto analysis.

RCA is guided by principles such as focusing on how and why problems occurred rather than who is at fault.

It often reveals multiple root causes requiring comprehensive solutions to prevent recurrence.

Effective RCA produces actionable insights for process improvement and risk mitigation.

It is used across industries such as software, manufacturing, healthcare, and engineering for quality control and safety.

It generally involves steps like problem identification, data collection, causal factor determination, root cause identification, and solution implementation.

It helps organizations reduce defects, enhance reliability, and avoid repeating mistakes.

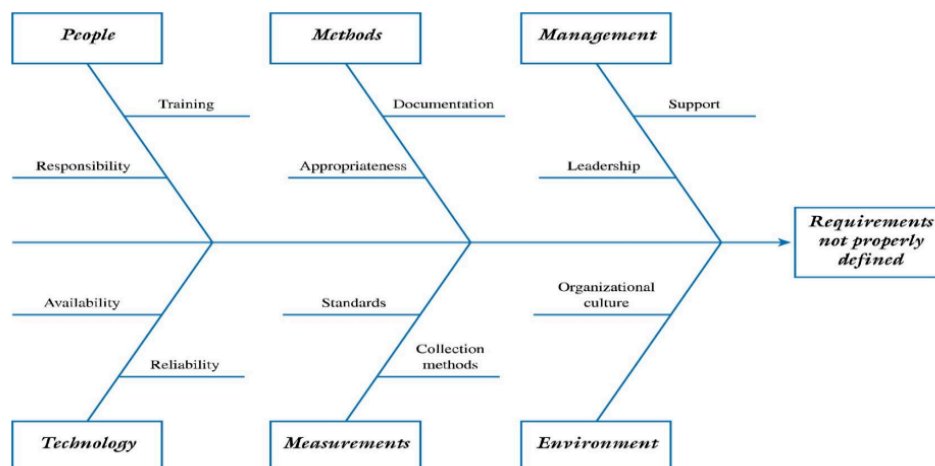
This approach leads to long-term quality improvement by addressing the source of problems rather than temporary fixes.

Software quality tools

Ishikawa Diagram

The Ishikawa Diagram, also known as the Fishbone or Cause-and-Effect Diagram, can be explained in points as follows:

- Developed by Kaoru Ishikawa in the 1960s as a quality control tool.
- Visualizes the causes of a specific problem or effect in a structured manner.
- The diagram resembles a fish skeleton, with the problem or effect at the "head" and potential causes as "bones" branching off.
- Helps identify, explore, and categorize root causes contributing to an issue.
- Often organized into main categories, commonly the "6 Ms": Manpower, Machines, Methods, Materials, Measurement, and Mother Nature (Environment).
- Used for root cause analysis, quality defect prevention, and process improvement.
- Facilitates brainstorming by visually sorting causes into meaningful groups.
- Supports team collaboration in diagnosing complex problems and prioritizing corrective actions.
- Widely employed in manufacturing, product development, IT, and business process management.
- A fundamental tool in techniques like Lean, Six Sigma, and continuous quality improvement.



Check list

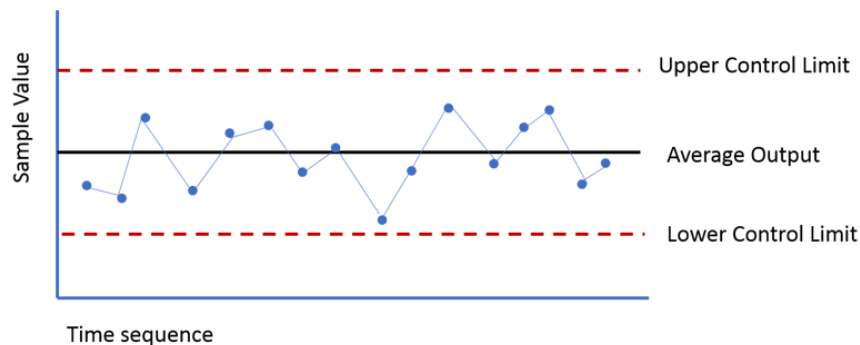
- A checklist is a structured list of items or tasks to be verified or completed in a systematic manner.
- Used to ensure consistency and completeness in processes, especially in quality assurance and testing.
- Helps verify that all necessary steps, standards, or criteria are met before proceeding.
- Acts as a reminder to prevent overlooking important aspects during inspections or reviews.
- Enhances communication among team members by providing a shared reference.
- Can be tailored for different stages such as requirements analysis, test planning, execution, and reporting.
- Improves productivity by streamlining repetitive verification processes.
- Facilitates documentation and accountability by recording completed checks.
- Supports risk reduction by systematically identifying omissions or defects.
- Widely used across software inspection, testing, audits, and compliance processes.

Overall, checklists are practical tools to maintain high quality, avoid errors, and improve process efficiency.

Control chart

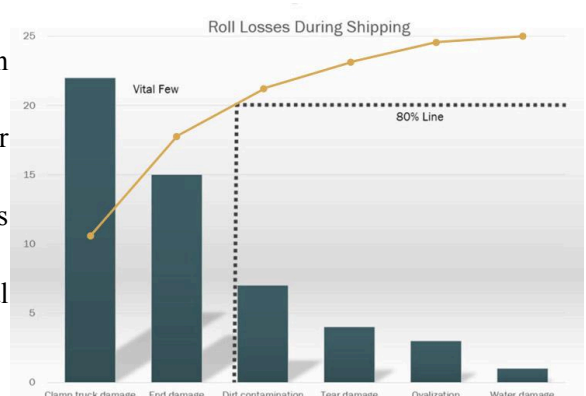
- A control chart is a graphical tool used in quality control to monitor how a process changes over time.
- It consists of a central line representing the process average, an upper control limit (UCL), and a lower control limit (LCL).
- Data points representing measurements are plotted in time order to detect variations.
- Helps distinguish between common cause variation (inherent in the process) and special cause variation (due to specific issues).
- Used to identify when a process is stable (in control) or unstable (out of control) for corrective action.
- Commonly applied in manufacturing, service industries, and software process control.
- Enables early detection of problems, helps reduce defects, and aids continuous process improvement.
- Types include X-bar charts, P charts, NP charts, and C charts, depending on data nature (variable or attribute).
- Control charts are key tools in statistical process control (SPC) and quality management methodologies like Six Sigma.
- Facilitate data-driven decisions by visualizing process performance trends and variability.

Thus, control charts provide an effective way to ensure process stability and product quality through continuous monitoring and timely interventions.



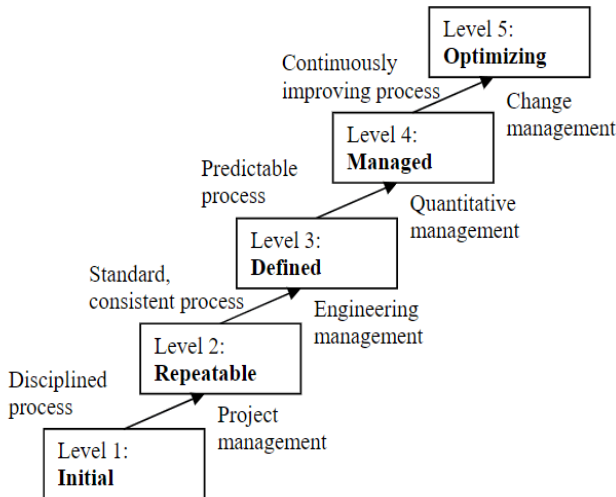
Pareto chart

- A Pareto chart is a bar graph combined with a line graph used in quality control and process improvement.
- It displays causes or factors in descending order of frequency or impact, with the largest bars on the left.
- The bars represent the frequency, cost, or magnitude of problems or defects.
- The line graph shows the cumulative percentage of the total impact, helping visualize the combined effect of the causes.



- Based on the Pareto principle (80/20 rule), it helps identify the "vital few" causes that contribute most to the problem.
- It is useful for prioritizing efforts on the most significant issues to maximize improvement.
- Helps teams focus on key problems rather than spreading resources thinly across all issues.
- Widely used in Six Sigma, Total Quality Management, and other quality methodologies.
- Facilitates clear communication of problem significance to stakeholders and management.
- Enables data-driven decision-making to solve problems effectively and efficiently.

The Pareto chart is a powerful tool to highlight and address the most impactful causes in any process or quality problem.



experiences guide new projects.

Level 3: Defined: Processes are documented, standardized, and integrated throughout the organization, ensuring consistency.

Level 4: Managed: Quantitative data and metrics are used to control processes and product quality predictably.

Level 5: Optimizing: Focuses on continuous process improvement through innovation and incremental advancements.

CMM helps organizations reduce risks, improve product quality, and enhance project predictability and efficiency. It is widely used as a benchmark for process maturity in software and systems engineering. This staged improvement model guides organizations to evolve from immature practices to optimized, measured, and continuously improving processes.

Principles of CMM

- People's capability is a competitive issue. Competition arises when different organizations are performing the same task (such as software development). In such a case, the people of an organization are sources of strategy and skills, which in turn results in better performance of the organization.
- The people's capability should be defined by the business objectives of the organization.
- An organization should invest in improving the capabilities and skills of the people as they are important for its success.
- The management should be responsible for enhancing the capability of the people in the organization.
- The improvement in the capability of people should be done as a process. This process should incorporate appropriate practices and procedures.

ISO 9001 (SQA Model)

The ISO 9000 family of quality management systems standards is designed to help organizations ensure that they meet the needs of customers and other stakeholders while meeting statutory and regulatory requirements related to a product or program. ISO 9000 deals with the fundamentals of quality management systems, including the seven quality management principles upon which the family of standards is based. ISO 9001 deals with the requirements that organizations wishing

SQA models

Capability Maturity Model (CMM)

CMM is a framework developed by the Software Engineering Institute (SEI) at Carnegie Mellon University aimed at improving software development processes. It provides a structured way for organizations to assess maturity and capability in their processes. The model consists of five maturity levels representing progressive stages of process improvement and control.

Level 1: Initial: Processes are ad hoc, chaotic, and unpredictable; success depends on individual effort.

Level 2: Repeatable: Basic project management processes are established to track cost, schedule, and functionality; previous

to meet the standard must fulfill.

- It sets out requirements for establishing, implementing, maintaining, and continuously improving a QMS to consistently meet customer and regulatory requirements.
- The model emphasizes a systematic approach based on documented processes covering the entire software lifecycle — from requirements to design, development, testing, delivery, and maintenance.
- Management responsibility is critical, requiring leadership commitment to quality policies, objective setting, and resource allocation.
- Focuses on customer satisfaction by ensuring product quality, on-time delivery, and compliance with contractual and legal requirements.
- Includes detailed requirements for resource management, competence of personnel, infrastructure, and work environment conducive to quality.
- Ensures traceability and control of documents, records, software releases, and configuration management.
- Certification to ISO 9001 demonstrates an organization's commitment to quality assurance, enhances efficiency, reduces defects, and builds customer confidence.
- Widely used to benchmark and improve software quality processes, aligning quality goals with business objectives and market needs.

This model helps organizations systematically manage software quality, ensuring reliable, consistent delivery that meets stakeholder expectations and regulatory demands.

Software total quality management

- STQM is a comprehensive management approach focused on achieving high-quality software by integrating quality principles throughout the software development lifecycle.
- It emphasizes continuous improvement, customer satisfaction, and involvement of all employees in quality-related activities.
- STQM promotes open communication channels and collection of customer feedback to understand and meet client needs effectively.
- Encourages active employee participation, fostering a culture where everyone contributes suggestions and takes ownership of quality.
- Aims to reduce software defects, waste, and inefficiencies by improving processes, standards, and practices continuously.
- STQM approaches quality from organizational, process, and product perspectives for holistic improvement.
- Focuses on process maturity and standardization to ensure consistent and reliable software quality.
- Encourages leadership commitment and resource allocation to maintain and enhance quality objectives.
- Enables organizations to deliver superior software products that satisfy end-user expectations and regulatory requirements.
- Draws from TQM principles in manufacturing, adapted for the nuances of software engineering and development.

In essence, STQM applies Total Quality Management philosophy specifically to software, integrating quality in every phase and involving all stakeholders for better software outcomes.

Six sigma

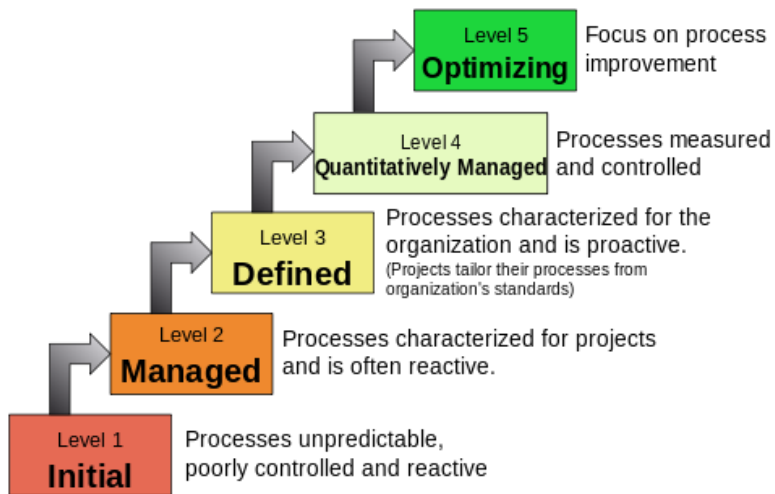
- Six Sigma is a data-driven methodology aimed at improving business processes by reducing defects and variability.
- It focuses on delivering near-perfect products and services to maximize customer satisfaction.
- The core principle is to identify and eliminate causes of errors using statistical and quality management tools.
- Six Sigma follows two main methodologies: DMAIC (for improving existing processes) and DMADV (for creating new processes or products).
- DMAIC stands for Define, Measure, Analyze, Improve, and Control:
 1. Define the problem and goals.

2. Measure current process performance.
 3. Analyze root causes of defects.
 4. Improve processes by implementing solutions.
 5. Control to sustain improvements.
- DMADV stands for Define, Measure, Analyze, Design, Verify, used for new product/process development.
 - Emphasizes a customer-centric approach by capturing the “voice of the customer.”
 - Uses various quality tools like root cause analysis, brainstorming, process mapping, and statistical analysis.
 - Six Sigma projects are led by trained professionals (e.g., Black Belts, Green Belts).
 - Widely adopted across industries for boosting process efficiency, reducing costs, and improving quality and reliability.

Six Sigma provides a structured, disciplined approach to achieve continuous process and quality improvement.

3 σ	6 σ
Five short or long landings at any major airport	One short or long landing in 10 years at all airports in the US
Approximately 1,350 poorly performed surgical operations in one week	One incorrect surgical operation in 20 years
Over 40,500 newborn babies dropped by doctors or nurses each year	Three newborn babies dropped by doctors or nurses in 100 years
Drinking water unsafe to drink for about 2 hours each month	Water unsafe to drink for one second every six years

Test Maturity Model (TMM): TMM is a framework designed to assess and improve the maturity of an organization's software testing processes.



- It is based on the Capability Maturity Model (CMM) and tailored specifically for testing.
- The model was developed by the Illinois Institute of Technology to provide structured guidance for testing process improvement.
- It provides a staged path with five maturity levels indicating progressive test process maturity and sophistication.
- Level 1 (Initial): Testing is informal, ad hoc, and unstructured with no defined processes or documentation.
- Level 2 (Definition): Testing is planned and documented with defined test strategies, plans, and

cases aligned to requirements.

- Level 3 (Integration): Testing processes are integrated into the software development lifecycle, typically performed after development.
- Level 4 (Management and Measurement): Testing activities are measured, managed, and controlled with defined quality criteria and metrics.
- Level 5 (Optimization): Testing processes are continuously optimized using tools and techniques focused on defect prevention and process improvement.
- TMM helps organizations identify weaknesses in testing, implement improvements, and achieve consistent, high-quality testing results.

This model guides organizations to elevate their testing capabilities from chaotic and reactive to optimized and preventive.